Паттерны проектирования
Часть 1

Грачев Д.Г.

Определение

Шаблон проектирования (*паттерн*, от англ. *design pattern*) — повторяемая архитектурная конструкция в сфере проектирования программного обеспечения, предлагающая решение проблемы проектирования в рамках некоторого часто возникающего контекста.

Обычно шаблон не является законченным образцом, который может быть прямо преобразован в код; это лишь пример решения задачи, который можно использовать в различных ситуациях. Объектно-ориентированные шаблоны показывают отношения и взаимодействия между классами или объектами, без определения того, какие конечные классы или объекты приложения будут использоваться.

"Низкоуровневые" шаблоны, учитывающие специфику конкретного языка программирования, называются идиомами. Это хорошие решения проектирования, характерные для конкретного языка или программной платформы, и потому не универсальные.

Типы

- Порождающие – отвечают за создание объектов.
- Структурные – определяют структуры классов и объектов.
- Паттерны корпоративных приложений
- Уровень предметной области (Domain Layer)
- Тестирование
- Отображение данных (Data Mapping)
- Паттерны работы с данными
- Распределенные и архитектурные паттерны
- Маршрутизация сообщений
- Развертывание
- Трансформация сообщений
- Конечные точки
- Управление каналами
- Декомпозиция
- Паттерны облачных приложений
- Коммуникация
- Транзакции и согласованность
- Наблюдаемость
- Поведенческие шаблоны
- И многие другие

Порождающие – отвечают за создание объектов.

Singleton

Гарантирует, что у класса есть только один экземпляр, и предоставляет глобальную точку доступа к нему.
(В данном случае как пример на базе класса, но для потокобезопасности лучше enum, так же обратите внимание на **synchronized** -> лучше использовать двойную проверку)

```
public class Singleton {

  private static Singleton instance = null;

  private Singleton() {
  }

  public synchronized Singleton getInstance() {
    if (instance == null) {
      instance = new Singleton();
    }
    return instance;
  }

}
```

Фабричный метод

Определяет интерфейс для создания объекта, но позволяет подклассам изменять тип создаваемого объекта.

```java
// Общий продукт
interface Product {
    void use();
}

// Конкретные продукты
class ConcreteProductA implements Product {
    public void use() { System.out.println("Using Product A"); }
}

class ConcreteProductB implements Product {
    public void use() { System.out.println("Using Product B"); }
}

// Создатель с фабричным методом
abstract class Creator {
    public abstract Product factoryMethod();

    public void someOperation() {
        Product product = factoryMethod();
        product.use();
    }
}
```

```java
// Конкретные создатели
class ConcreteCreatorA extends Creator {
    public Product factoryMethod() { return new ConcreteProductA(); }
}

class ConcreteCreatorB extends Creator {
    public Product factoryMethod() { return new ConcreteProductB(); }
}

// Использование
public class Main {
    public static void main(String[] args) {
        Creator creator = new ConcreteCreatorA();
        creator.someOperation(); // Using Product A
    }
}
```

Абстрактная фабрика

Предоставляет интерфейс для создания семейств взаимосвязанных объектов без указания их конкретных классов

```java
// Абстрактные продукты
interface Button {
    void render();
}

interface Checkbox {
    void check();
}

// Конкретные продукты для Windows
class WinButton implements Button {
    public void render() { System.out.println("Render Windows button"); }
}

class WinCheckbox implements Checkbox {
    public void check() { System.out.println("Check Windows checkbox"); }
}

// Конкретные продукты для Mac
class MacButton implements Button {
    public void render() { System.out.println("Render Mac button"); }
}

class MacCheckbox implements Checkbox {
    public void check() { System.out.println("Check Mac checkbox"); }
}
```

```java
// Абстрактная фабрика
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

// Конкретные фабрики
class WinFactory implements GUIFactory {
    public Button createButton() { return new WinButton(); }
    public Checkbox createCheckbox() { return new WinCheckbox(); }
}

class MacFactory implements GUIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}
```

Абстрактная фабрика

```java
// Клиентский код
public class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        button = factory.createButton();
        checkbox = factory.createCheckbox();
    }

    public void paint() {
        button.render();
        checkbox.check();
    }

    public static void main(String[] args) {
        Application app = new Application(new WinFactory());
        app.paint();
    }
}
```

Как видите, стоит определить только в одном месте с какой именно платформой работает система, а далее обработка происходит однообразно без зависимости в логике от платформы. В реальном проекте может быть сотни функций на создание элементов!

Builder (Строитель). Пример простого класса.

```java
public class Computer {  3 usages  new *
    private String cpu;  3 usages
    private String ram;  3 usages
    private String storage;  3 usages

    public String getCpu() {  no usages  new *
        return cpu;
    }

    public void setCpu(String cpu) {  1 usage  new *
        this.cpu = cpu;
    }

    public String getRam() {  no usages  new *
        return ram;
    }

    public void setRam(String ram) {  1 usage  new *
        this.ram = ram;
    }

    public String getStorage() {  no usages  new *
        return storage;
    }

    public void setStorage(String storage) {  1 usage  new *
        this.storage = storage;
    }

    @Override  new *
    public String toString() {
        return "Computer{" +
                "cpu='" + cpu + '\'' +
                ", ram='" + ram + '\'' +
                ", storage='" + storage + '\'' +
                '}';
    }
}
```

Заставляет писать много кода. Ответственность на разработчике.

```java
public class Main {  new *

    private static void printComputer(Computer computer) {
        System.out.println("Computer: " + computer);
    }

    public static void main(String[] args) {  new *
        Computer computer = new Computer();
        computer.setCpu("intel");
        computer.setRam("16gb");
        computer.setStorage("hdd");
        printComputer(computer);
    }
}
```

Builder (Строитель). Пример использования.

```java
// Продукт
class Computer {
    private String cpu;
    private String ram;
    private String storage;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
    }

    // Геттеры
    public String getCpu() { return cpu; }
    public String getRam() { return ram; }
    public String getStorage() { return storage; }

    @Override
    public String toString() {
        return "Computer [cpu=" + cpu + ", ram=" + ram + ", storage=" + storage + "]";
    }

    // Статический вложенный класс Builder
    public static class Builder {
        private String cpu;
        private String ram;
        private String storage;

        public Builder setCpu(String cpu) { this.cpu = cpu; return this; }
        public Builder setRam(String ram) { this.ram = ram; return this; }
        public Builder setStorage(String storage) { this.storage = storage; return this; }

        public Computer build() {
            return new Computer(this);
        }
    }
}
```

Отделяет конструирование сложного объекта от его представления, позволяя создавать разные представления.

```java
// Использование
public class Main {
    public static void main(String[] args) {
        Computer computer = new Computer.Builder()
                .setCpu("Intel i7")
                .setRam("16GB")
                .setStorage("512GB SSD")
                .build();
        System.out.println(computer);
    }
}
```

Lombok

```java
// Продукт
class Computer {
    private String cpu;
    private String ram;
    private String storage;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
    }

    // Геттеры
    public String getCpu() { return cpu; }
    public String getRam() { return ram; }
    public String getStorage() { return storage; }

    @Override
    public String toString() {
        return "Computer [cpu=" + cpu + ", ram=" + ram + ", storage=" + storage + "]";
    }

    // Статический вложенный класс Builder
    public static class Builder {
        private String cpu;
        private String ram;
        private String storage;

        public Builder setCpu(String cpu) { this.cpu = cpu; return this; }
        public Builder setRam(String ram) { this.ram = ram; return this; }
        public Builder setStorage(String storage) { this.storage = storage; return this; }

        public Computer build() {
            return new Computer(this);
        }
    }
}
```

Пишет код за вас!

```java
14  */
15  @ToString  3 usages  new *
16  @Builder
17  public class Computer {
18      private String cpu;
19      private String ram;
20      private String storage;
21  }
22
```

Prototype (Прототип)

```java
// Класс, реализующий клонирование
class Shape implements Cloneable {
    private String type;
    private int x, y;

    public Shape(String type, int x, int y) {
        this.type = type;
        this.x = x;
        this.y = y;
    }

    // Геттеры/сеттеры
    public String getType() { return type; }
    public int getX() { return x; }
    public int getY() { return y; }

    // Клонирование через переопределение clone()
    @Override
    public Shape clone() {
        try {
            return (Shape) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Cloning failed", e);
        }
    }

    @Override
    public String toString() {
        return "Shape [type=" + type + ", x=" + x + ", y=" + y + "]";
    }
}
```

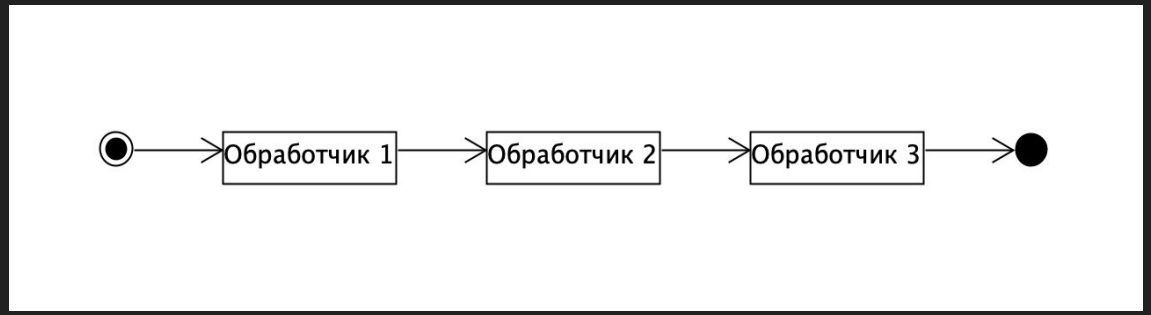
Cloneable является неявным ребенком Object, который имеет метод copy

Создает новые объекты путем клонирования существующих (прототипов).

```java
// Использование
public class Main {
    public static void main(String[] args) {
        Shape original = new Shape("Circle", 10, 20);
        Shape clone = original.clone();

        System.out.println("Original: " + original);
        System.out.println("Clone   : " + clone);
        System.out.println("Same object? " + (original == clone)); // false
    }
}
```

Поведенческие паттерны

Цепочка обязанностей




```java
// Базовый класс обработчика
abstract class SupportHandler {
    protected SupportHandler nextHandler;

    public void setNext(SupportHandler handler) {
        this.nextHandler = handler;
    }

    public abstract void handle(String issue);
}
```

```java
// Использование
public class Main {
    public static void main(String[] args) {
        // Строим цепочку
        SupportHandler level1 = new Level1Handler();
        SupportHandler level2 = new Level2Handler();
        SupportHandler level3 = new Level3Handler();

        level1.setNext(level2);
        level2.setNext(level3);

        // Тестируем
        System.out.println("Запрос 1:");
        level1.handle("простой вопрос");

        System.out.println("\nЗапрос 2:");
        level1.handle("средняя проблема");

        System.out.println("\nЗапрос 3:");
        level1.handle("критический баг");
    }
}
```

Цепочка обязанностей

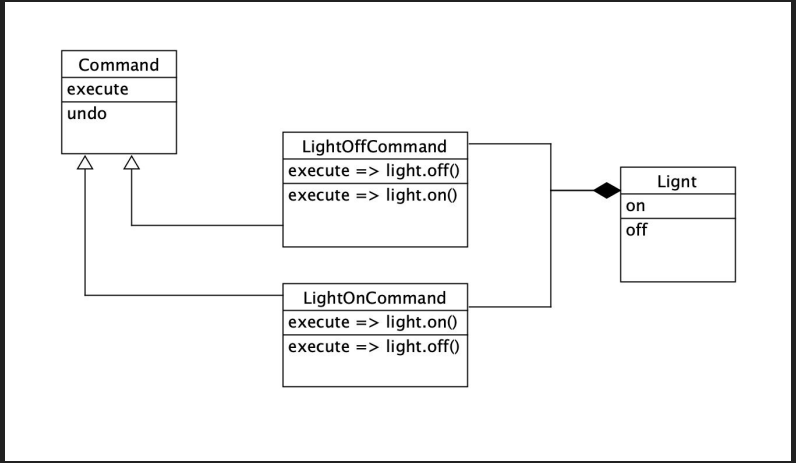
Как задавали в main: у каждого обработчика есть знание кому передать далее. Соответственно, когда запрос поступает, он может выполнить свою часть работы (или не выполнять вовсе) и далее передать следующему на обработку. Основная цель: иметь контролируемые цепочки ответственности, вместо бесконечных if-else-then.

```java
// Конкретные обработчики
class Level1Handler extends SupportHandler {
    public void handle(String issue) {
        if (issue.equals("простой вопрос")) {
            System.out.println("Level 1: Решил простой вопрос");
        } else if (nextHandler != null) {
            System.out.println("Level 1: Передаю дальше");
            nextHandler.handle(issue);
        }
    }
}

class Level2Handler extends SupportHandler {
    public void handle(String issue) {
        if (issue.equals("средняя проблема")) {
            System.out.println("Level 2: Решил среднюю проблему");
        } else if (nextHandler != null) {
            System.out.println("Level 2: Передаю дальше");
            nextHandler.handle(issue);
        }
    }
}

class Level3Handler extends SupportHandler {
    public void handle(String issue) {
        System.out.println("Level 3: Решаю сложную проблему: " + issue);
    }
}
```

Command(Команда)

```java
// Интерфейс команды
interface Command {
    void execute();
    void undo();
}

// Получатель (устройство, которым управляем)
class Light {
    public void on() { System.out.println("Свет включен"); }
    public void off() { System.out.println("Свет выключен"); }
}

// Конкретные команды
class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() { light.on(); }
    public void undo() { light.off(); }
}

class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() { light.off(); }
    public void undo() { light.on(); }
}
```

Инкапсулируем действие в объект. Можно выполнять команды, отменять их, ставить в очередь. В рамках логики вы выполняете команды, а они уже в свою очередь скрывают логику реализации. Может показаться избыточным, но позволяет экономить время и ресурсы (DRY/YAGNI), тем самым упрощая поддержку.



Command(Команда)

```java
// Вызыватель (пульт)
class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }

    public void pressUndo() {
        command.undo();
    }
}
```

```java
// Использование
public class Main {
    public static void main(String[] args) {
        Light light = new Light();
        Command lightOn = new LightOnCommand(light);
        Command lightOff = new LightOffCommand(light);

        RemoteControl remote = new RemoteControl();

        remote.setCommand(lightOn);
        remote.pressButton();  // Свет включен
        remote.pressUndo();    // Свет выключен

        remote.setCommand(lightOff);
        remote.pressButton();  // Свет выключен
        remote.pressUndo();    // Свет включен
    }
}
```

Interpreter (Интерпретатор)

```java
// Использование
public class Main {
    public static void main(String[] args) {
        // Выражение: 10 + 5 - 3
        Expression expression = new SubtractExpression(
            new AddExpression(
                new NumberExpression(10),
                new NumberExpression(5)
            ),
            new NumberExpression(3)
        );

        System.out.println("Результат: " + expression.interpret()); // 12
    }
}
```

Интерпретируем простой язык. Пример: парсинг и выполнение арифметических выражений.

Interpreter (Интерпретатор)

```java
// Интерфейс выражения
interface Expression {
    int interpret();
}

// Число
class NumberExpression implements Expression {
    private int number;

    public NumberExpression(int number) {
        this.number = number;
    }

    public int interpret() {
        return number;
    }
}
```

```java
// Сложение
class AddExpression implements Expression {
    private Expression left;
    private Expression right;

    public AddExpression(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    public int interpret() {
        return left.interpret() + right.interpret();
    }
}

// Вычитание
class SubtractExpression implements Expression {
    private Expression left;
    private Expression right;

    public SubtractExpression(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    public int interpret() {
        return left.interpret() - right.interpret();
    }
}
```

Iterator (Итератор)

```
// Интерфейс итератора
interface Iterator {
    boolean hasNext();
    String next();
}
```

Перебираем элементы коллекции, не зная её внутреннего устройства.

Итератор

```java
// Простая коллекция
class StringCollection {
    private String[] items = new String[10];
    private int count = 0;

    public void add(String item) {
        if (count < items.length) {
            items[count++] = item;
        }
    }

    // Создаём итератор для коллекц
    public Iterator iterator() {
        return new StringIterator();
    }

    // Внутренний класс - итератор
    private class StringIterator implements Iterator {
        private int index = 0;

        public boolean hasNext() {
            return index < count;
        }

        public String next() {
            return items[index++];
        }
    }
}
```

```java
// Интерфейс итератора
interface Iterator {
    boolean hasNext();
    String next();
}

// Использование
public class Main {
    public static void main(String[] args) {
        StringCollection collection = new StringCollection();
        collection.add("Java");
        collection.add("Python");
        collection.add("C++");

        Iterator iterator = collection.iterator();
        while (iterator.hasNext()) {
            System.out.println("Язык: " + iterator.next());
        }
    }
}
```

Практические задачи

Задание 1. Singleton: «Один экземпляр на всех»

Суть: Создать класс, который может существовать только в одном экземпляре, и проверить, что при нескольких попытках получения объекта мы всегда получаем один и тот же.

Что нужно сделать:

- Создать класс `Logger` (логирует сообщения). У него должен быть приватный конструктор и статический метод `getInstance()`, возвращающий единственный экземпляр.
- Добавить метод `log(String message)`, который выводит в консоль `[LOG]` + сообщение.
- В методе `main` попытаться получить два экземпляра логгера и сравнить их (через `==`), а затем вызвать метод `log` у любого из них.

```
Пример вывода:
Получен экземпляр логгера.
Ещё один вызов getInstance()
вернул тот же объект? true
[LOG] Привет, мир!
```

Практические задачи

Задание 2. Factory Method: «Фабрика приветствий»

Суть: Есть интерфейс `Greeting` с методом `sayHello()`. Разные классы реализуют его по-своему (английский, французский, испанский). Фабрика создаёт нужный объект в зависимости от переданной строки.

Что нужно сделать:

- Интерфейс `Greeting` с методом `void sayHello()`.
- Классы `EnglishGreeting`, `FrenchGreeting`, `SpanishGreeting` — каждый выводит своё приветствие в консоль.
- Класс `GreetingFactory` с методом `createGreeting(String language)`, который возвращает соответствующий объект.
- В `main` создать фабрику, запросить приветствие для `"english"` и вызвать `sayHello()`.

Пример вывода:

```
Hello!
```

Практические задачи

Задание 3. Abstract Factory: «Светлая и тёмная темы»

Суть: Есть два типа элементов интерфейса: кнопка и фон. В зависимости от темы (светлая/тёмная) они имеют разный цвет. Абстрактная фабрика создаёт целое семейство объектов.

Что нужно сделать:

- Интерфейсы `Button` и `Background` с методами `void display()`.
- Конкретные классы `LightButton`, `DarkButton`, `LightBackground`, `DarkBackground`. В `display()` они выводят своё название (например, `"Светлая кнопка"`).
- Интерфейс `ThemeFactory` с методами `Button createButton()` и `Background createBackground()`.
- Реализации `LightThemeFactory` и `DarkThemeFactory`.
- В `main` выбрать фабрику (например, `LightThemeFactory`) и создать с её помощью кнопку и фон. Вызвать у них `display()`.

```
Пример вывода:
Светлая кнопка

Светлый фон
```

Практические задачи

Задание 4. Builder: «Собираем пиццу»

Суть: У пиццы есть обязательные параметры (тесто, размер) и необязательные (сыр, соус, добавки). Билдер позволяет удобно задавать только нужные поля и в конце получить готовый объект.

Что нужно сделать:

- Класс `Pizza` с полями: `dough`, `size`, `cheese` (boolean), `sauce` (String). Все поля `final`.
- Внутри класса `Pizza` создать статический вложенный класс `Builder`. У него такие же поля, но не `final`. Методы `setDough()`, `setSize()`, `addCheese()`, `setSauce()` возвращают `this`. Метод `build()` создаёт объект `Pizza`.
- В `main` с помощью билдера создать пиццу: тесто "тонкое", размер "средняя", с сыром, соус "томатный". Вывести полученную пиццу (переопределить `toString()`).

```
Пример вывода:
Pizza [dough=тонкое, size=средняя, cheese=true, sauce=томатный]
```

Практические задачи

Задание 5. Prototype: «Клонирование круга»

Суть: Есть класс `Circle` с полями `radius` и `color`. Нужно научиться создавать точную копию объекта (клон), чтобы изменения в клоне не затрагивали оригинал.

Что нужно сделать:

- Класс `Circle` реализует интерфейс `Cloneable` и переопределяет метод `clone()` (сделать `public`).
- Добавить поля `int radius`, `String color`, конструктор, геттеры/сеттеры и метод `toString()`.
- В `main` создать оригинальный круг, затем клонировать его. Изменить радиус клона. Вывести оба круга и сравнить их через `==` и через `equals` (или просто показать, что они разные).

Пример вывода:

```
Оригинал: Circle [radius=10, color=red]

Клон: Circle [radius=20, color=red]

Оригинал после изменения клона: Circle [radius=10, color=red]

Оригинал и клон - разные объекты? true
```

Подписывайтесь

Ресурсы

- https://ru.wikipedia.org/wiki/%D0%A8%D0%B0%D0%B1%D0%BB%D0%BE%D0%BD_%D0%BF%D1%80%D0%BE%D0%B5%D0%BA%D1%82%D0%B8%D1%80%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D1%8F - вики
- https://www.ozon.ru/product/head-first-patterny-proektirovaniya-2-e-izdanie-frimen-erik-robson-elizabet-380155226/?at=z6tOq2VpqcN4G0PJfPEOXZ1C2gv3VGczooVg5uyWvlBz - **Паттерны проектирования. 2-е издание | Фримен Эрик, Робсон Элизабет**
- **https://habr.com/ru/articles/113995/ - цепочка обязаностей**
- **https://habr.com/ru/articles/114455/ - Команда**